UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA
DEPARTAMENTO
DE INDUSTRIAS

SQL 2/2

LABORATORIO 5

ICN-292 — Sistemas de Información para la Gestión
Segundo Semestre 2026 - Campus Vitacura
Repositorio Github :https://github.com/antarpizarro-blip/ICN292-Lab5-Pizarro-Antar.git

Profesores: José Miguel González Paul - José Luis Sáez Tamayo - Patricio Cifuentes
Estudiante: Antar Pizarro 21330781-9 202260576-6

Resumen

En el siguiente laboratorio, se evalúa el uso de SQL como mecanismo de registro y plataforma para consultas sobre una base de datos. En este caso se trabaja bajo una base de datos de la compañía Andes Bike, la cual dispone de 4 sedes, 8 trabajadores y 8 modelos de bicicletas diferentes. Se poseen 4 tablas consolidadas con la información a ingresar en la base de datos, Tiendas, Trabajadores, Bicicletas y Ventas. Estas 4 tablas son generadas dentro de SQL para luego ingresar todos los datos y registros dentro de cada una, a partir de estos datos se realizaron 10 encuestas que estudian relaciones y comportamientos según tiendas, trabajadores, modelos etc.

Consultas

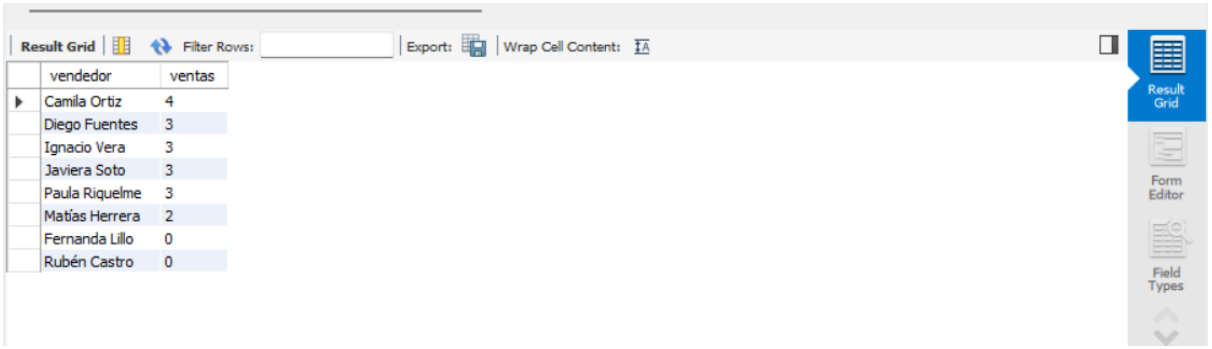
1. Liste todos los vendedores con la cantidad de ventas que registraron, incluidos los que no registraron ninguna. Ordene de mayor a menor cantidad y, a igualdad, por nombre. Los que no tienen ventas deben mostrar cero, no quedar en blanco.

Código:

```
USE andesbike;

SELECT ve.nombre AS vendedor,
COUNT(v.id_venta) AS ventas
FROM vendedores ve
LEFT JOIN ventas v ON ve.id_vendedor = v.id_vendedor
GROUP BY ve.id_vendedor, ve.nombre
ORDER BY ventas DESC, ve.nombre;
```

Respuesta:



vendedor	ventas
Camila Ortiz	4
Diego Fuentes	3
Ignacio Vera	3
Javiera Soto	3
Paula Riquelme	3
Matías Herrera	2
Fernanda Lillo	0
Rubén Castro	0

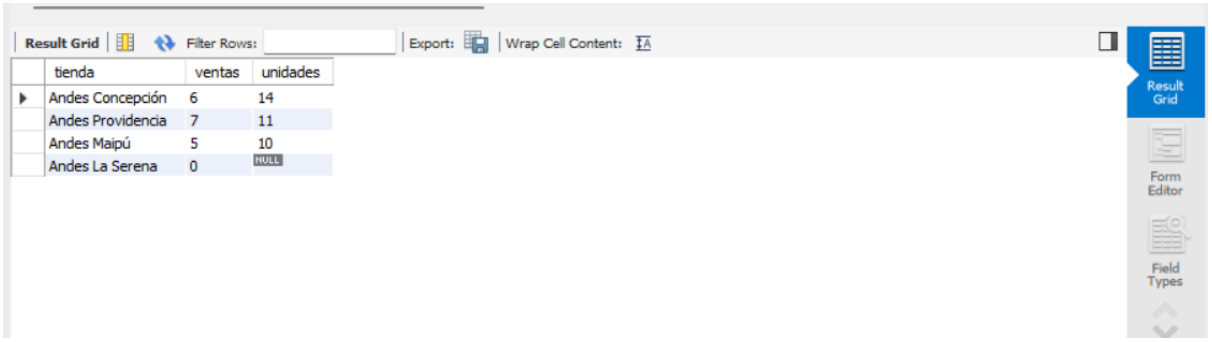
2. Para cada tienda muestre su nombre, cuántas ventas se registraron y cuántas unidades se vendieron, incluidas las tiendas que no tienen ningún vendedor. Ordene por unidades de mayor a menor.

Código:

```
USE andesbike;

SELECT t.nombre AS tienda,
COUNT(v.id_venta) AS ventas,
Sum(v.unidades) AS unidades
FROM tiendas t
LEFT JOIN vendedores ve ON t.id_tienda = ve.id_tienda
LEFT JOIN ventas v ON ve.id_vendedor = v.id_vendedor
GROUP BY t.id_tienda, t.nombre
ORDER BY unidades DESC;
```

Resultado:



tienda	ventas	unidades
Andes Concepción	6	14
Andes Providencia	7	11
Andes Maipú	5	10
Andes La Serena	0	NULL

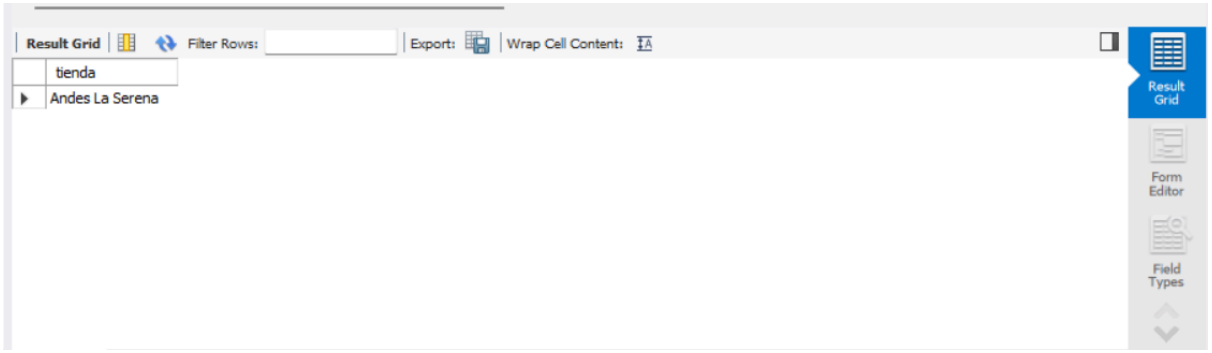
3. Liste las tiendas que no tienen ningún vendedor asignado.

Código:

```
USE andesbike;

SELECT t.nombre AS tienda
FROM tiendas t
LEFT JOIN vendedores v ON t.id_tienda = v.id_tienda
WHERE v.id_vendedor IS NULL
```

Resultado:



tienda
Andes La Serena

4. Liste los modelos que no se vendieron ni una sola vez en junio de 2025. Explique en una línea en qué parte de la consulta puso la condición del mes y qué habría devuelto si la hubiera puesto en la otra.

Código:

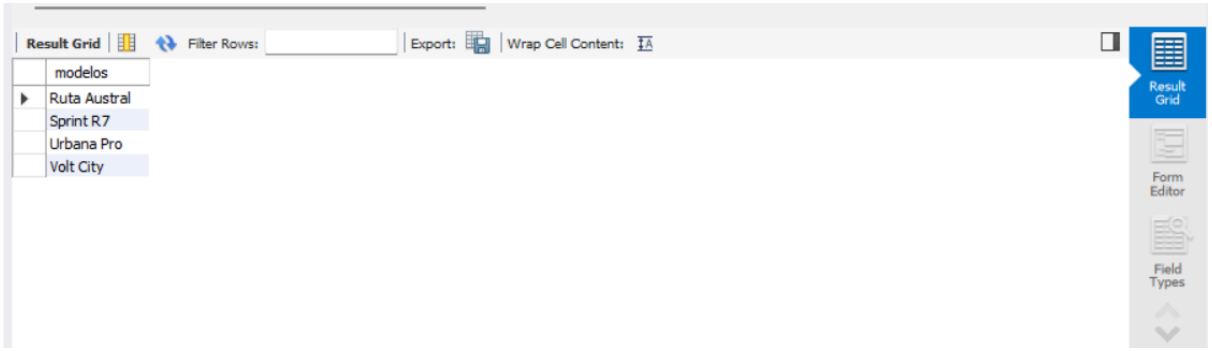
```
USE andesbike;

SELECT b.modelo AS modelos
FROM bicicletas b
LEFT JOIN ventas v ON b.id_bicicleta = v.id_bicicleta
AND v.fecha >= '2025-06-01'
AND v.fecha <= '2025-06-30'
WHERE v.id_venta IS NULL
ORDER BY b.modelo;
```

La selección de fecha se realiza en la condición ON del LEFT JOIN, ya que de esta manera se consideran solo las ventas realizadas en el mes de junio. Si se condiciona el periodo de fechas dentro del WHERE de la consulta, el

LEFT JOIN se comportaría como un INNER JOIN y los modelos no vendidos serían ignorados por ende la consulta no entregaría las bicicletas no vendidas.

Resultado:



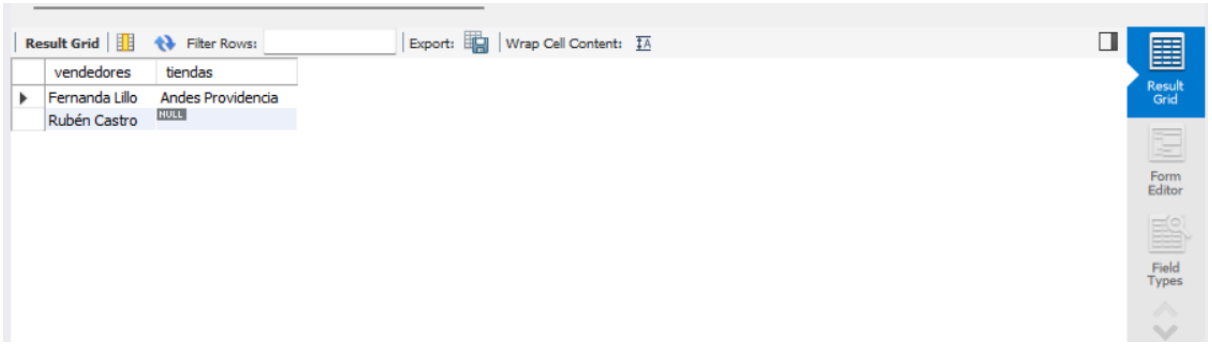
5. Liste los vendedores que no han registrado ninguna venta, mostrando también el nombre de su tienda. Ninguno de ellos puede quedar fuera del resultado.

Código:

```
USE andesbike;

SELECT ve.nombre AS vendedores,
t.nombre AS tiendas
FROM vendedores ve
LEFT JOIN ventas v ON ve.id_vendedor = v.id_vendedor
LEFT JOIN tiendas t ON ve.id_tienda = t.id_tienda
WHERE v.id_venta IS NULL
ORDER BY vendedores;
```

Resultado:



6. Liste los pares de vendedores que trabajan en la misma tienda, poniendo primero al que ingresó antes. Muestre los dos nombres, las dos fechas de ingreso y la tienda. Cada par debe aparecer una sola vez.

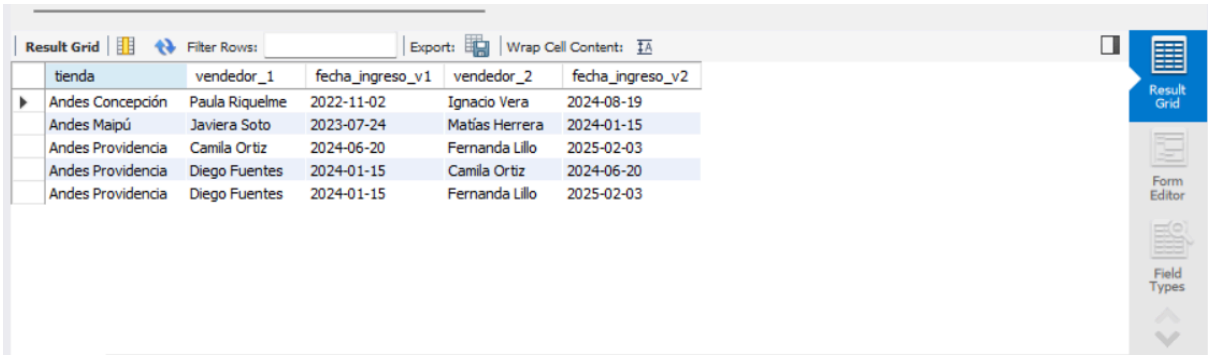
Código:

```
USE andesbike;

SELECT t.nombre AS tienda,
v1.nombre AS vendedor_1,
```

```
v1.fecha_ingreso AS fecha_ingreso_v1,
v2.nombre AS vendedor_2,
v2.fecha_ingreso AS fecha_ingreso_v2
FROM vendedores v1
JOIN vendedores v2 ON v1.id_tienda = v2.id_tienda AND v1.fecha_ingreso < v2.fecha_ingreso
JOIN tiendas t ON v1.id_tienda = t.id_tienda
ORDER BY tienda;
```

Resultado:



tienda	vendedor_1	fecha_ingreso_v1	vendedor_2	fecha_ingreso_v2
Andes Concepción	Paula Riquelme	2022-11-02	Ignacio Vera	2024-08-19
Andes Maipú	Javiera Soto	2023-07-24	Matías Herrera	2024-01-15
Andes Providencia	Camila Ortiz	2024-06-20	Fernanda Lillo	2025-02-03
Andes Providencia	Diego Fuentes	2024-01-15	Camila Ortiz	2024-06-20
Andes Providencia	Diego Fuentes	2024-01-15	Fernanda Lillo	2025-02-03

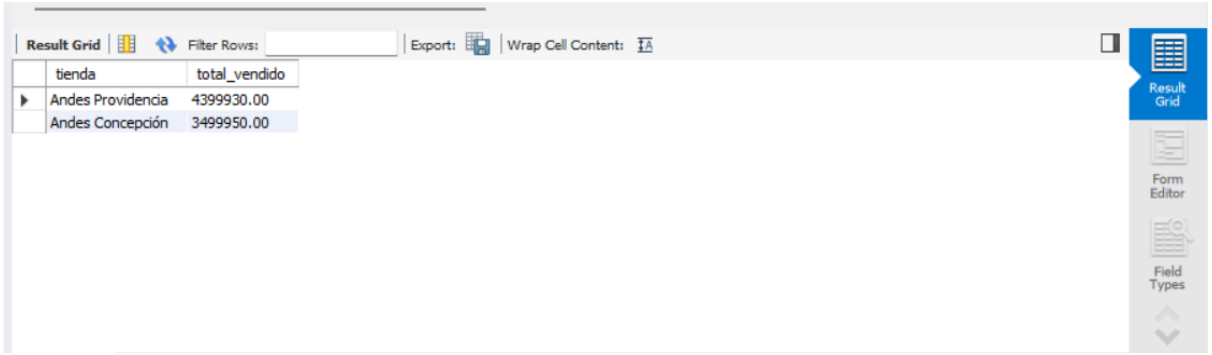
7. Para cada tienda, calcule el monto total vendido en bicicletas de las categorías 'Montaña' y 'Ruta', dejando solo las tiendas que superan los 2.000.000. Ordene de mayor a menor.

Código:

```
USE andesbike;

SELECT t.nombre AS tienda,
SUM(v.unidades * b.precio) AS total_vendido
FROM tiendas t
JOIN vendedores ve ON t.id_tienda = ve.id_tienda
JOIN ventas v ON ve.id_vendedor = v.id_vendedor
JOIN bicicletas b ON v.id_bicicleta = b.id_bicicleta
WHERE b.categoria IN('Ruta', 'Montaña')
GROUP BY t.id_tienda, t.nombre
HAVING total_vendido > 2000000
ORDER BY total_vendido DESC;
```

Resultado:



tienda	total_vendido
Andes Providencia	4399930.00
Andes Concepción	3499950.00

8. Liste el modelo y el precio de las bicicletas cuyo precio supera el promedio del catálogo, de mayor a menor. Indique en el informe el valor de ese promedio y de dónde lo obtuvo.

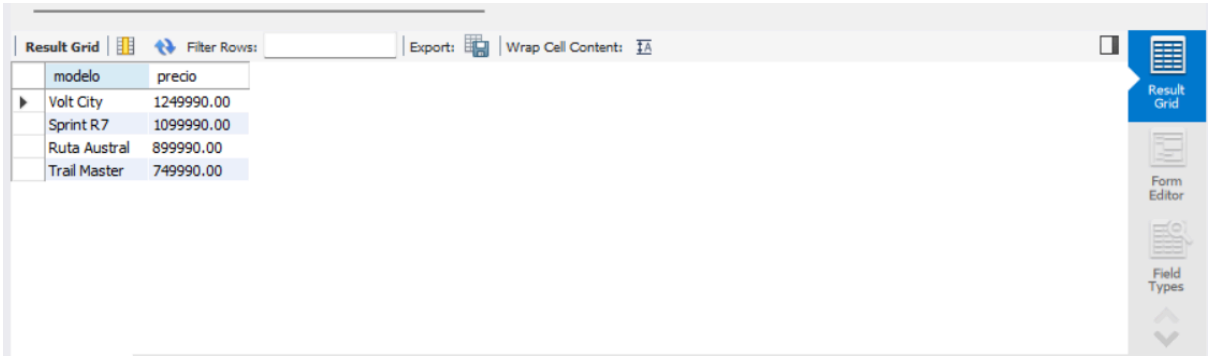
Código:

```
USE andesbike;

SELECT modelo,
precio
FROM bicicletas
WHERE precio > (SELECT AVG(precio) FROM bicicletas)
ORDER BY precio DESC;
```

El precio promedio es 657.490,00 y en el código se obtiene dentro del WHERE, específicamente cuando se usa AVG(precio).

Resultado:



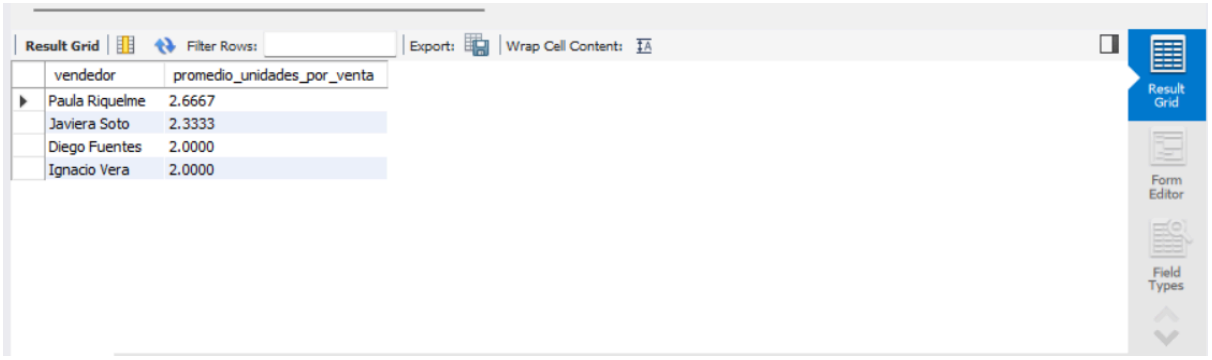
9. Liste los vendedores cuyo promedio de unidades por venta supera el promedio de unidades de todas las ventas de la cadena. Muestre el nombre y su promedio, de mayor a menor y, a igualdad, por nombre.

Código:

```
USE andesbike;

SELECT ve.nombre AS vendedor,
AVG(v.unidades) AS promedio_unidades_por_venta
FROM vendedores ve
JOIN ventas v ON ve.id_vendedor = v.id_vendedor
GROUP BY ve.id_vendedor, ve.nombre
HAVING AVG(v.unidades) > (SELECT AVG(unidades) FROM ventas)
ORDER BY promedio_unidades_por_venta DESC, ve.nombre;
```

Resultado:



vendedor	promedio_unidades_por_venta
Paula Riquelme	2.6667
Javiera Soto	2.3333
Diego Fuentes	2.0000
Ignacio Vera	2.0000

10. Liste todos los vendedores con el monto total que vendieron, incluidos los que no vendieron nada, de mayor a menor monto. Explique en una línea por qué algunos vendedores no muestran cero y qué tendría que agregar a la consulta para que lo mostraran

Código:

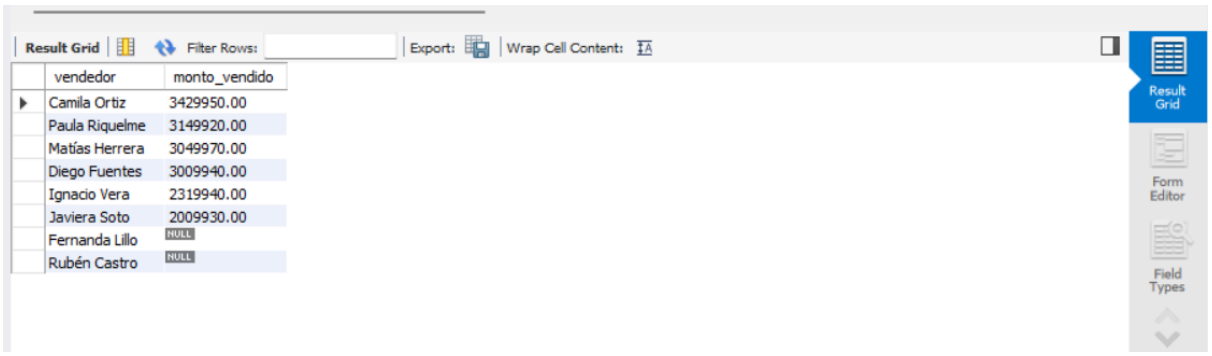
```
USE andesbike;

SELECT ve.nombre AS vendedor,
SUM(v.unidades * b.precio) AS monto_vendido
FROM vendedores ve
LEFT JOIN ventas v ON ve.id_vendedor = v.id_vendedor
LEFT JOIN bicicletas b ON v.id_bicicleta = b.id_bicicleta
GROUP BY ve.id_vendedor, ve.nombre
ORDER BY monto_vendido DESC;
```

Para dos de los vendedores, en el monto vendido se devuelve NULL, esto es porque no tienen registro de ventas dentro de la tabla de ventas, por ende el programa no logra ejecutar la suma de las ventas que se solicita en el comando SUM, entregando un valor nulo en lugar de “0”. Para recibir el valor cero se debe usar otro comando como puede ser COALESCE, el cual retorna el primer valor de una lista de comandos que sea no nulo, esto permite ingresar una opción de respuesta diferente en caso de que el valor del comando deseado sea nulo, en este caso la 4ta línea de código donde se promptéa el comando SUM quedaría de la siguiente forma:

```
COALESCE(SUM(v.unidades * b.precio), 0) AS monto_vendido
```

Resultado:



vendedor	monto_vendido
Camila Ortiz	3429950.00
Paula Riquelme	3149920.00
Matías Herrera	3049970.00
Diego Fuentes	3009940.00
Ignacio Vera	2319940.00
Javiera Soto	2009930.00
Fernanda Lillo	NULL
Rubén Castro	NULL